

PRÁCTICA 03

STACK MEAN

Sistema de Gestión de Empleados

Patrones de Diseño de APIs

Maestría de Software

ESTUDIANTE	Carlos Adrian Espinoza Alvarez
ASIGNATURA	Patrones de Diseño de APIs
DOCENTE	Ing. Patsy Prieto, MSc.
FECHA	23 de septiembre de 2026
REPOSITORIO	github.com/AdrianEspinoza92/PatronesDisenoApis-Practica-3

Objetivo

Diseñar e implementar una arquitectura web desacoplada, robusta y escalable utilizando MongoDB, Express, Angular y Node.js sobre TypeScript.

1. Resumen de la solución

Transformé el CRUD base en una aplicación MEAN con separación estricta de responsabilidades, validación perimetral, respuestas HTTP uniformes y estado reactivo. Incorporé una colección de Postman con pruebas automáticas para los contratos principales.

Reto	Implementación	Evidencia
1	Repository e inversión de dependencias	Controlador sin imports de Mongoose
2	DTO Zod, Response Wrapper y errores globales	400/404 uniformes y fieldErrors
3	BehaviorSubject y referencias inmutables	employees\$ de solo lectura
4	Componentes Smart/Dumb	@Input, @Output y async pipe

Arquitectura general

Angular	REST / JSON	Express	IEmployeeRepository	MongoDB
Smart + Dumb	Wrapper	Controller	Adapter Mongoose	usuarios_db

Tecnologías verificadas

Backend	Node.js, Express 5, TypeScript, Mongoose, Zod, Morgan, CORS
Frontend	Angular 20, RxJS, HttpClient, componentes standalone
Persistencia	MongoDB Community 8.0.32 en 127.0.0.1:27017
Pruebas	Postman/Newman: 10 solicitudes y 20 aserciones aprobadas

2. Reto 1 - Desacoplamiento del backend

Implementé la inversión de dependencias mediante una abstracción recibida por constructor. Confiné Mongoose al adaptador de persistencia, de modo que la capa HTTP desconoce el ODM.

Contrato de persistencia

```
export interface IEmployeeRepository {
  findAll(page: number, limit: number): Promise<EmployeePage>;
  findById(id: string): Promise<Employee | null>;
  create(input: CreateEmployee): Promise<Employee>;
  update(id: string, input: UpdateEmployee): Promise<Employee | null>;
  delete(id: string): Promise<boolean>;
}
```

Inyección en el controlador

```
export class EmployeeController {
  constructor(private readonly repository: IEmployeeRepository) {}

  create = async (_request, response, next) => {
    const employee = await this.repository.create(response.locals.body);
    response.status(201).json(successResponse('Empleado creado', employee));
  };
}
```

Comprobación	Resultado
Import de mongoose en controllers/	No existe
Import de EmployeeModel en controllers/	No existe
CRUD REST ejecutado contra MongoDB local	Correcto
Paginación GET ?page=1&limit=20	Correcto

Con esta estructura puedo sustituir Mongoose por otra fuente de datos mediante un nuevo adaptador que implemente la interfaz, sin modificar el controlador.

3. Reto 2 - Validación y respuesta universal

Implementé esquemas Zod para validar req.body, req.params y req.query antes de ejecutar la lógica de negocio. Transferí los datos validados mediante res.locals para mantener compatibilidad con Express 5.

Entrada inválida probada en Postman

```
POST /api/v1/employees
{
  "nombre": "An",
  "cargo": "Developer",
  "departamento": "TI",
  "sueldo": -200
}
```

Respuesta 400 Bad Request

```
{
  "success": false,
  "message": "Solicitud inválida",
  "data": null,
  "errors": {
    "fieldErrors": {
      "nombre": ["Debe contener al menos 3 caracteres"],
      "departamento": ["Debe contener al menos 3 caracteres"],
      "sueldo": ["El sueldo no puede ser negativo"]
    }
  }
}
```

Caso	HTTP	Wrapper
Creación válida	201	success: true, data: empleado
Body inválido	400	success: false, errors.fieldErrors
ID inválido	400	success: false, message uniforme
Empleado inexistente	404	success: false, data: null
Ruta inexistente	404	interceptor global

Resultado de Newman: 10 solicitudes ejecutadas, 20 aserciones aprobadas y 0 fallos.

4. Evidencia de funcionamiento

La interfaz Angular que desarrollé consume la API REST y presenta los empleados persistidos en MongoDB. El componente Smart obtiene employees\$ mediante async pipe y delega formulario y tabla a componentes Dumb.

STACK MEAN · PRÁCTICA 03

Gestión de empleados

Arquitectura desacoplada, validada y reactiva.

Nuevo empleado

Nombre

Cargo

Departamento

Sueldo

0

Guardar

Personal registrado

NOMBRE	CARGO	DEPARTAMENTO	SUELDO	ACCIONES
María Torres	Líder de Producto	Operaciones	\$3,900.00	Editar Eliminar
Carlos Ramírez	Analista de Datos	Tecnología	\$3,200.00	Editar Eliminar
Andrea Mendoza	Arquitecta de Software	Innovación	\$4,500.00	Editar Eliminar
Andrea Mendoza	Arquitecta de Software	Innovación	\$4,800.00	Editar Eliminar

Figura 1. Aplicación Angular ejecutándose en `http://localhost:4200` con cuatro registros persistidos.

Verificación final	Estado
npm run check - backend	APROBADO
npm run build - backend	APROBADO
npm run build - frontend	APROBADO
MongoDB ping	APROBADO
Colección Postman/Newman	20/20

5. Pruebas y conclusiones

Pruebas realizadas

Prueba	Resultado obtenido
Creación de empleado	201 Created y persistencia en MongoDB
Consulta paginada	200 OK con items, total, page, limit y pages
Consulta y actualización por ID	200 OK con datos consistentes
Body e ID inválidos	400 Bad Request generado por Zod
Recurso o ruta inexistente	404 con Response Wrapper
Eliminación	200 OK y confirmación del identificador

Conclusiones

Con la implementación completé los cuatro retos solicitados. Desacoplé el backend del ODM mediante Repository, protegí el perímetro HTTP con Zod y un esquema universal de respuesta, y organicé el frontend con estado reactivo y separación Smart/Dumb.

Las pruebas demostraron que las operaciones CRUD mantienen respuestas consistentes y que los datos inválidos son rechazados antes de llegar a la lógica de negocio. La compilación estricta de TypeScript y las 20 aserciones automatizadas aprobadas respaldan la reproducibilidad de la solución.